Database Design and Migrations

Contents

Database Design and Migrations **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 2
- References . . . 2
- Related . . . 2

Database Design and Migrations

Scope

Schema conventions, constraints and the migration process. Engine: **PostgreSQL, hosted on Supabase** — `brain/decisions/ADR-0006-datastore.md`.

Official requirements

- Subject V.3: all service data is stored server-side; the backend is the guardian of truth.
- Subject V.2: vote and playlist-reorder race conditions must be handled — this is a schema concern first and an application concern second.
- Subject IV.1: a fresh clone must be able to bring its dependencies up automatically, so the schema is created by a scripted, committed migration run.
- PostgreSQL documented behaviour for unique indexes, `xmin` concurrency and transaction isolation levels.

Project policy

Naming

Object	Convention
table	plural `snake_case` — `playlist_tracks`
column	`snake_case`
primary key	`id`
foreign key	`<referenced_singular>_id`
index	`ix_<table>_<columns>`
unique index	`ux_<table>_<columns>`
check constraint	`ck_<table>_<rule>`

Schema rules

- Every table has `id`, `created_at_utc`, and `updated_at_utc`.

- Deletions of user-owned aggregates are **soft** (`deleted_at_utc`) so an audit trail survives.
- Foreign keys are declared with an explicit `ON DELETE` action. No orphan rows.
- Constraints that protect an invariant live in the database, not only in C#:
  - `ux_votes_event_id_track_id_user_id` — enforces one vote per user per track.
  - `ux_users_email` — enforces unique accounts.
  - concurrency token column on `playlists` and `devices`.
- Playlist ordering uses a sparse integer or decimal `position` with gaps, so an insert between two tracks does not rewrite the whole list.
- Foreign keys and frequently filtered columns are indexed. Every index has a reason recorded in the migration.
- No secrets, no API keys, no real personal data in seed scripts.

Supabase specifics

- The schema is owned by **EF Core migrations**, never by the Supabase dashboard. A dashboard edit desynchronises migration history and is treated as an incident.
- Two connection strings: the pooled one (Supavisor) for the API, the **direct** one for running migrations. Migrations through the transaction pooler fail on prepared statements.
- Application tables live in the `public` schema. Never touch Supabase's internal schemas (`auth`, `storage`, `realtime`) — we do not use those features.
- Row Level Security is enabled with deny-by-default policies as defence in depth. It is never the only authorisation gate; the application checks ownership and licence itself.
- No client ever connects to the database directly. Only the backend holds a connection string.

Migrations

- EF Core migrations are the only way the schema changes. No manual SQL against a shared database.
- One migration per logical change, named `<Verb><Noun>` — `AddVoteUniqueIndex`.
- Review the generated SQL before committing. Generated migrations are a draft, not an answer.
- Every migration must be **reversible**, or its irreversibility must be stated in `brain/migrations/migrations-index.md` together with the rollback plan.
- Destructive changes (drop column, drop table, narrow a type) go through expand → migrate → contract:
  1. add the new shape,
  2. backfill and switch readers/writers,
  3. remove the old shape in a later migration.
- Data backfills are separate from schema migrations and idempotent.
- Record each migration in `brain/migrations/migrations-index.md`.

Checklist

- ☐ Invariants enforced by constraints, not only by code
- ☐ Unique index on votes exists before the vote endpoint ships
- ☐ Concurrency token on every hotspot row
- ☐ Generated SQL reviewed, migration reversible or rollback documented
- ☐ Migration row added to the Brain index

References

- EF Core migrations — https://learn.microsoft.com/ef/core/managing-schema/migrations/
- Expand/contract pattern — https://martinfowler.com/bliki/ParallelChange.html

Related

`../../10-backend/ef-core-persistence/STANDARD.md` · `brain/architecture/domain-model.md`